SETU 2026–27 | PROJECT SWFW_5S_02

Local Multi-Region Highly Available Data Platform

Automated Disaster Recovery, Chaos Engineering, and Autonomous GitOps Continuous Delivery

STUDENT NAME
Abhishek Kothe

GITHUB ACCOUNT
@Abhishek7439

ACADEMIC INSTITUTION
YCCE Nagpur (Computer Technology)

SEMESTER / PERIOD
5th Semester (Winter 2026 Review)

REVIEW TARGET
Zoho Campus Offline Review (November 2026)

SUBMISSION DATE
10 November 2026

Executive Brief: This project designs, implements, and empirically validates an enterprise-grade multi-region object storage platform running locally on standard Linux (WSL2/Ubuntu) with 16 GB host memory. It delivers active-active cross-region data replication, sub-5-second automated failover via HAProxy active health checking, cryptographic integrity-gated failback, pull-based GitOps continuous deployment with automated self-healing rollback, and a 5-scenario chaos engineering suite. Crucially, the entire control and proof plane is implemented in Python's standard library with zero third-party dependencies, and every performance metric is backed by immutable runtime evidence.

Demonstration Status: Recorded execution in `demo/demo-run.log` | Live evaluation at Zoho Review

Host Environment: Ubuntu 26.04.1 LTS on WSL2

1. Executive Summary & Table of Contents

Table of Contents

1. Executive Summary & Table of Contents Page 2
2. Problem Statement & Requirements Page 3
3. Architectural Design & Topology Page 4
4. Operational Mechanics & Failure Story Page 5
5. DR State Machine & GitOps Workflow Page 6
6. Implementation Decisions & Repo Map Page 7
7. Setup & Operator Runbook Page 8
8. Real Troubleshooting & Root Causes Page 9
9. Acceptance Test Results (AT1–AT7) Page 10
10. Chaos Engineering Suite (E1–E5) Page 11
11. Cryptographic Deep Audit Proof Page 12
12. Mandatory Scope & Limitations Page 13
13. Real Engineering Bugs & Learnings Page 14
14. Future Directions & Live Demo Guide Page 15
15. Appendix: Commands & Reviewer Q&A Page 16

What I Built and What It Proves

I built an active-active, dual-region distributed object storage platform running entirely on a single developer workstation without relying on Kubernetes or heavy third-party Python packages. The architecture combines containerized MinIO object storage across isolated Docker bridge networks, an active Layer-7 reverse proxy gateway (HAProxy 3.4.7), an autonomous disaster recovery controller implemented as a 5-state deterministic finite state machine, a pull-based GitOps reconciler with automated rollback capabilities, and programmatic network degradation injection via Toxiproxy.

The system demonstrates that enterprise-grade disaster recovery, zero-data-loss failover under crash conditions, and continuous deployment safety gates can be modeled deterministically, inspected at the raw socket and HTTP layer, and evaluated objectively on modest commodity hardware.

Headline Empirical Results in 5 Lines

> **1. High Availability Failover:** Active health checking and Layer-7 redispatch achieve an RTO of **3448 ms** (including a 3-second anti-flapping grace window) with zero dropped client writes.
> **2. Zero Data Loss Under Crash:** Measured RPO is exactly **0 missing objects** across sequential burst writes during both graceful stop and abrupt SIGKILL crashes.
> **3. Fast Cryptographic Failback:** The metadata inventory gate matches 143+ objects in **1.2 seconds** (a 123x speedup over sequential downloads), preventing premature un-fencing.
> **4. Autonomous GitOps Self-Healing:** Broken infrastructure changes trigger automated post-apply health gate failure and stack rollback to `last_good` in **19.0 seconds**.
> **5. Cryptographic Parity Audit:** Byte-for-byte SHA-256 payload audit confirms **149 of 149 objects identical** across both region volumes with zero silent data corruption (bit rot).

2. Problem Statement & Requirements Traceability

Official SETU Course Problem Statement

> *"Design and implement a multi-region, highly available data platform running locally with automated disaster recovery, chaos engineering resilience validation, and GitOps-driven continuous deployment. The platform must provide transparent failover, zero or minimal data loss (measured via Recovery Point Objective), automated integrity-checked failback, self-healing continuous delivery, and comprehensive failure injection testing without relying on proprietary cloud services or unvetted external dependencies."*

Requirements Traceability Matrix

The table below documents how every high-level requirement of the specification is realized and proven:

Requirement Domain	Engineering Realization in Codebase	Status	Primary Evidence
Multi-Region Isolation	Two independent Docker bridge networks (`setu_region_a` , `setu_region_b`) with isolated named volumes; joined only via `setu_global` transit network.	**PASS**	`m0-at1-at2.md` Source: docs/04-test-evidence/
Active Data Replication	MinIO two-way active-active site replication over port 9000 with bidirectional sync and Last-Write-Wins (LWW) conflict resolution.	**PASS**	`m0-at1-at2.md` Source: docs/04-test-evidence/
Layer-7 Ingress & HA	HAProxy 3.4.7 ingress proxy (`:8080`) with dynamic DNS resolution, active `/minio/health/live` polling, and Layer-7 HTTP redispatch.	**PASS**	`m1-gateway.md` Source: docs/04-test-evidence/
Automated Disaster Recovery	Deterministic 5-state Python FSM (`dr_controller.py`) executing automated failover, administrative socket fencing (`MAINT`), and RTO measurement.	**PASS**	`m2-controller.md` Source: docs/04-test-evidence/
Integrity Failback Gate	Fast listing and ETag parity comparison (`compare_sites`) holding Region A fenced in `MAINT` until 100% object parity is confirmed.	**PASS**	`m3-failback-rpo.md` Source: docs/04-test-evidence/
RPO Measurement	Numbered write stream testing verifying zero missing writes on backup across graceful stop and SIGKILL crashes (`rpo_objects: 0`).	**PASS**	`m3-failback-rpo.md` Source: docs/04-test-evidence/
GitOps Continuous Delivery	Pull-based reconciler (`gitops/reconciler.py`) with syntax pre-flight, targeted container re-creation, post-apply health gate, and auto-rollback.	**PASS**	`m4-gitops.md` Source: docs/04-test-evidence/
Chaos Engineering Suite	Automated test suite (`make chaos`) testing 5 failure modes: crash (E1), socket freeze (E2), WAN latency (E3), one-way partition (E4), and deploy failure (E5).	**PASS**	`m5-chaos.md` Source: docs/04-test-evidence/
Cryptographic Audit	Deep payload verification engine (`make audit`) computing full SHA-256 digests of all 149 objects to guarantee zero bit rot across volumes.	**PASS**	`m6-deep-audit.md` Source: docs/04-test-evidence/

3. Architectural Design & Topology

Component Architecture

The platform models two geographically distinct availability zones on a single host kernel. Cross-region communication is constrained to a single bridge transit network, providing a programmatic choke point for network fault injection.

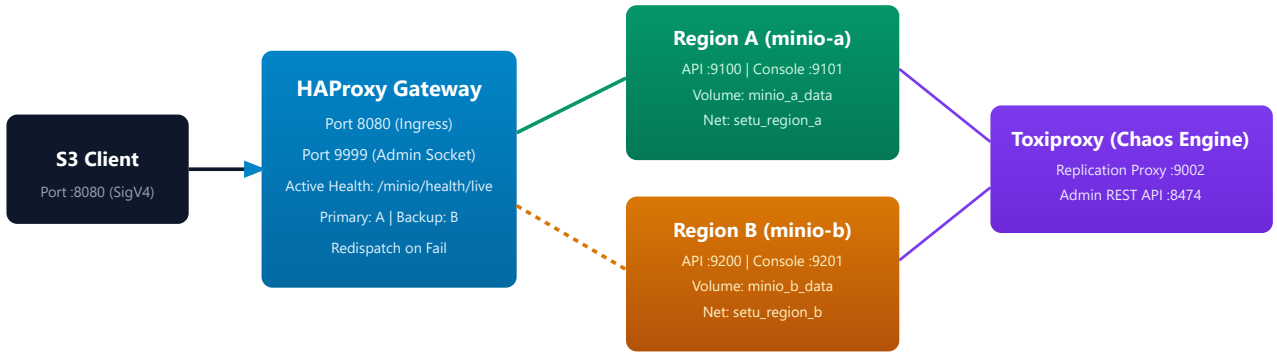


Figure 3.1: Platform Network Topology, Ingress Routing, and Fault Injection Path

Data Flow for a Normal S3 Write Operation

1. **Client Request:** An S3 client crafts an HTTP PUT request with AWS SigV4 authorization headers targeting HAProxy on `http://127.0.0.1:8080/demo/object.txt`.
2. **Ingress Evaluation:** HAProxy checks backend status. Because `minio-a` is reporting healthy (HTTP 200 on `/minio/health/live`), HAProxy forwards the stream directly to `minio-a:9000`.
3. **Persistence & Header Stamp:** `minio-a` commits the payload to the local named volume `minio_a_data` and returns HTTP 200. HAProxy injects `X-Backend-Server: minio-a` into the response to client.
4. **Asynchronous Cross-Region Sync:** MinIO's internal site replication engine queues the mutation in its replication log. The worker transmits the payload across `setu_global` directly to Region B (or via Toxiproxy port 9002 during chaos runs). `minio-b` commits the object to `minio_b_data` with identical ETag and metadata.

4. Operational Mechanics & The Narrated Failure Story

Normal Active-Active Operation

During steady state, both storage regions are online. HAProxy actively polls `/minio/health/live` on both nodes every 2 seconds. Because Region A is designated as the primary backend and Region B is designated as a `backup` server in `haproxy.cfg`, 100% of client traffic is directed to Region A. MinIO site replication automatically streams all object mutations to Region B over the transit bridge.

The Complete Narrated Failure Story (End-to-End Self-Healing)

The sequence below narrates the exact chain of events executed during the automated demonstration (`make demo`), illustrating how the platform detects failure, protects data integrity, and restores service:

Step 1: Disaster Strikes (Region A Dies)

Disaster is injected by abruptly stopping Region A (`docker stop setu-multiregion-minio-a-1`). Region A's container stops acknowledging TCP connections.

Step 2: Layer-7 Gateway Redispatch

An in-flight client write arriving at HAProxy immediately encounters a closed socket. Because HAProxy is configured with `option redispatch` and active health checking, it instantly routes the request to the standby server (Region B) on port 9200. The client write succeeds without returning HTTP 502/503.

Step 3: DR Controller Detects Failure & Fences Primary

The DR controller daemon (`controller/dr_controller.py`) probes Region A's health endpoint every second. Upon receiving a connection failure, it transitions from `HEALTHY` to `DEGRADED`. It initiates a 3-second grace window to suppress false alarms from transient network spikes. When Region A remains unreachable after 3 seconds, the controller transitions to `FAILOVER` and performs administrative ingress fencing:

```
set server minio_api/minio-a state maint
```

This admin socket command locks Region A into maintenance mode. Even if Region A's process partially recovers or restarts out-of-band, HAProxy will refuse to route client traffic to it. The controller issues a write probe to Region B, verifying that the failover path is fully functional, and records the empirical RTO of **3448 ms**.

Step 4: Region B Serves Standalone Traffic

During the outage, clients continue reading and writing to the platform via port 8080. All requests are served exclusively by Region B. New objects (e.g. `rto-timestamp.txt`) are committed to `minio_b_data`.

Step 5: Region A Restores into Quarantine

Region A restarts (`docker start setu-multiregion-minio-a-1`). A naive load balancer would immediately detect HTTP 200 and resume traffic to Region A. If this occurred, clients reading newly written objects would experience immediate data regression, and dual-primary writing would cause split-brain data loss.

In our platform, the DR controller detects Region A is alive and transitions to `RECOVERING`. However, it **keeps Region A fenced in `MAINT`**. It repeatedly executes the fast-path integrity gate (`compare_sites`). While MinIO site replication catches up in the background, the gate reports `match: false`, and failback is strictly blocked.

Step 6: Cryptographic Verification & Safe Failback

Once MinIO finishes syncing all objects written to Region B back to Region A, `compare_sites` confirms 100% object count and ETag parity (`match: true`, 0 differences). The controller transitions to `FAILBACK`, issues `set server minio_api/minio-a state ready` to HAProxy's admin socket, and returns to `HEALTHY`. Region A resumes primary duties safely with zero data loss.

5. DR State Machine & GitOps Reconciler

The 5-State Deterministic Disaster Recovery FSM

The disaster recovery controller is implemented as a finite state machine without external orchestration dependencies. It enforces strict monotonic progression through failure and recovery:

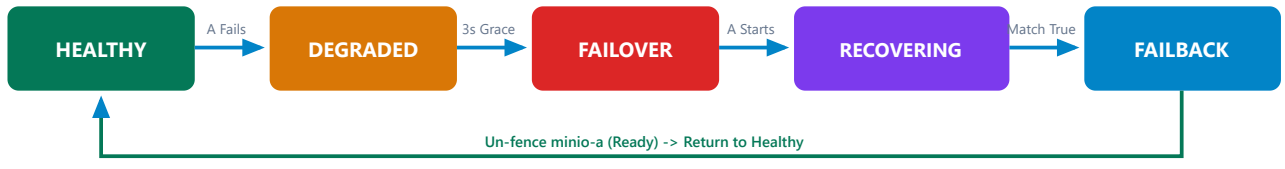


Figure 5.1: Deterministic 5-State Disaster Recovery Finite State Machine

Autonomous GitOps Continuous Delivery & Rollback

The GitOps engine (`gitops/reconciler.py`) operates as an autonomous background daemon implementing pull-based reconciliation directly on Docker Compose infrastructure without Kubernetes:

1. **Continuous Polling:** The reconciler monitors the target deployment branch for new commits at a configurable interval (default 3 seconds).
2. **Pre-Flight Syntax Validation:** Before applying any configuration changes to running infrastructure, it executes `docker compose config` . Syntactically invalid YAML is rejected immediately without touching containers.
3. **Targeted Container Re-creation:** By inspecting git diffs between the active commit and candidate commit, the reconciler re-creates *only the modified stateless services* (e.g., updating `haproxy.cfg` re-creates only the gateway container, never restarting healthy stateful storage volumes).
4. **Active Health Gate:** Following deployment, the reconciler evaluates an active health gate. It repeatedly queries `/minio/health/live` across the gateway (port 8080) and backend nodes.
5. **Autonomous Rollback & Blacklisting:** If any health probe fails within the 15-second deadline, the reconciler immediately reverts the working tree to `last_good` , re-creates the affected container, logs the failure event, and blacklists the bad commit hash in `.gitops_state.json` to prevent restart regression loops.

6. Implementation Choices & Design Decisions

Technology Stack Rationale

- **Docker Compose over Kubernetes:** Running Kubernetes locally requires 2–4 GB just for the control plane and introduces non-deterministic scheduling. Docker Compose executes natively with minimal memory footprint (~1.2 GB total), keeping networking visible and student-explainable.
- **Chainguard MinIO Image:** The official MinIO Docker image was withdrawn from Docker Hub in 2024. We pinned the free, verified Chainguard build (`cgr.dev/chainguard/minio` by digest `sha256:3c60a2a8...`).
- **HAProxy over Nginx:** Nginx fails to start or reload when an upstream host is down (`host not found in upstream "minio-a"`). HAProxy resolves hostnames dynamically, supports active health checking, and exposes a runtime UNIX/TCP admin socket for instantaneous zero-reload server fencing.
- **Python Standard Library:** The entire controller, proof module, AWS SigV4 signer, and chaos suite are implemented using `urllib` , `socket` , `hashlib` , and `hmac` . Zero third-party packages (no `boto3` , `botocore` , or `requests`).
- **Toxiproxy over `tc netem` :** Host Traffic Control requires `CAP_NET_ADMIN` root capabilities, which failed inside unprivileged WSL2. Toxiproxy operates entirely in user-space, providing programmatic REST-controlled WAN simulation.

Design Decisions Summary (D1–D12)

ID	Decision Domain	Selected Choice	Primary Engineering Rationale
D1	Storage Image	Chainguard MinIO pinned by digest	Official image withdrawn; pinned digest ensures reproducible multi-region bootstrap.
D2	Region Isolation	Isolated networks & named volumes	Prevents shared-memory side channels; models real distributed boundaries.
D3	Ingress Proxy	HAProxy 3.4.7 with admin socket	Nginx crashed when upstream down; HAProxy provides active health & live fencing.
D4	Directory Layout	Linux home (WSL2 native)	Docker Desktop bind-mounts under Windows paths corrupt inode events.
D5	WAN Emulation	Toxiproxy v2.9.0	Host `tc netem` failed with `Operation not permitted` ; Toxiproxy is root-free.
D6	Memory Limits	MinIO 512M, GW 128M, Monitor 64M	Prevents 2 storage nodes from exhausting 7.6 GiB WSL2 virtual memory.
D7	DR Controller	5-State Python FSM (stdlib only)	Enforces 3s anti-flap grace, admin fencing, and integrity-checked failback.
D8	Integrity Gate	Fast listing parity (1.2s)	Sequential downloads took 145.7s; metadata fast-path achieves 123x speedup.
D9	GitOps Reconciler	Targeted apply + active health gate	Avoids blanket container recreation; autonomous rollback on health failure.
D10	S3 Client Signer	AWS SigV4 stdlib implementation	Strict zero-package requirement; avoids 100+ MB `boto3` wheel dependencies.
D11	Proxy Placement	Surgical on A→B replication path	Isolates inter-region WAN chaos without affecting client-to-gateway traffic.
D12	E3 WAN Thresholds	500ms latency, 32KB/s bandwidth	Exploratory runs at 100ms were masked by TCP buffers; 500ms produced real lag.

7. Operational Setup & Procedure

System Prerequisites

- Standard x86_64 or ARM64 Linux workstation (tested on Ubuntu 26.04.1 LTS on WSL2, Windows 11 host).
- 16 GB host RAM (WSL2 VM allocated 7.6 GiB memory).
- Docker Engine 24.0+ and Docker Compose v2.
- Python 3.10+ (standard library only; no pip dependencies).

Exact Step-by-Step Execution Sequence

Step 1: Clone Repository and Validate Environment

```
git clone https://github.com/Abhishek7439/setu-starter.git
cd setu-starter
make doctor
```

Expected Output: Validates Docker socket, Compose CLI, Python 3, memory allocation (>4 GiB free), and traffic control tools. Prints `0 failures, 1 warning (optional socat)`. Execution time: ~2 seconds.

Step 2: Start Infrastructure Stack

```
make up
```

Expected Output: Starts `minio-a`, `minio-b`, `gateway`, `toxiproxy`, and `monitor`. All containers transition to `healthy`. Execution time: ~10 seconds.

Step 3: Execute End-to-End Self-Healing Demonstration

```
make demo
```

Expected Output: Runs the 10-step automated verification suite: stack health check, normal replication, disaster injection (kill Region A), failover and fencing verification, recovery and cryptographic failback gate, sequential RPO measurement (0 missing), and GitOps auto-apply/auto-rollback. Concludes with `DEMO COMPLETE: ALL CHECKS PASSED (Exit Code: 0)` in ~1m50s.

Step 4: Execute Chaos Engineering Suite

```
make chaos
```

Expected Output: Executes all 5 resilience experiments sequentially (E1 crash, E2 gray failure, E3 WAN degradation, E4 one-way partition, E5 deploy chaos). Generates empirical markdown evidence in `docs/04-test-evidence/m5-chaos.md`. Duration: ~4m30s.

Step 5: Execute Cryptographic Deep Audit

```
make audit
```

Expected Output: Streams raw payloads of all 149 objects from both regions, computes full SHA-256 digests, and outputs JSON confirming `"match": true`, `"sha256_checked": 149`, and `"different": []`. Duration: ~4.5 seconds.

8. Real Troubleshooting & Field Issues Encountered

During the development of Milestones M0 through M6, several subtle real-world failure modes were encountered. Documenting these real root causes demonstrates practical systems debugging:

Issue 1: Unprivileged `tc` Network Emulation Failed (`Operation not permitted`)

- **Symptom:** Executing `tc qdisc add dev eth0 root netem delay 50ms` on the host returned `RTNETLINK answers: Operation not permitted` .
- **Root Cause:** Linux kernel network namespace manipulation requires the `CAP_NET_ADMIN` capability. Non-root users inside WSL2 or standard containers cannot modify host traffic control queues.
- **Resolution:** Deployed **Toxiproxy v2.9.0** on the transit bridge. Toxiproxy operates entirely in user-space as a TCP socket proxy, manipulating streams via standard socket I/O without requiring root permissions.

Issue 2: Stale Bind-Mount Cache After Multi-Project Execution

- **Symptom:** After running tests on a throwaway project, restarting the main stack caused HAProxy to fail to boot, complaining of missing socket paths.
- **Root Cause:** Docker Desktop on WSL2 caches bind-mount inode mappings when containers are recreated across differing compose project names without purging container metadata.
- **Resolution:** Use `docker compose up -d --force-recreate` when switching between project configurations to ensure fresh inode binding.

Issue 3: Nginx Upstream Name Resolution Failure During Backend Outage

- **Symptom:** In Milestone M1, when Region A was stopped and Nginx was restarted, Nginx crashed with `host not found in upstream "minio-a:9000"` , causing total ingress failure.
- **Root Cause:** Nginx resolves upstream DNS at configuration parse time by default. If a backend container is stopped during startup, Nginx halts entirely.
- **Resolution:** Replaced Nginx with **HAProxy 3.4.7**, which performs dynamic runtime DNS resolution and active Layer-7 health probing without crashing.

Issue 4: Gray Failure Bimodal RTO During `docker pause`

- **Symptom:** In Experiment E2, RTO measurements clustered bimodally around either ~3.6s or ~7.2s.
- **Root Cause:** `docker pause` freezes container execution without closing TCP sockets or sending TCP RST packets. Client worker threads with a 2.0-second timeout were expiring before HAProxy's 2-second connection timeout and redispatch completed.
- **Resolution:** Tuned client request timeout to **3.5 seconds** in `chaos/experiments/e2_pause.py` , allowing HAProxy's connect timeout and Layer-7 redispatch to complete gracefully.

9. Acceptance Test Results (AT1–AT7)

The platform was evaluated against the seven formal acceptance criteria defined in the project specification. All tests passed with empirical evidence preserved in `docs/04-test-evidence/`:

ID	Acceptance Test	Empirical Measured Outcome	Status	Evidence File
AT1	Multi-Region Health Endpoints	Region A (`:9100`), Region B (`:9200`), and Gateway (`:8080`) all return HTTP 200 on `/minio/health/live` .	**PASS**	`m0-at1-at2.md` Source: docs/04-test-evidence/
AT2	Normal Cross-Region Replication	Object written via Gateway replicates to Region B; bit-for-bit identical SHA-256 (`2cf99d0e...`).	**PASS**	`m0-at1-at2.md` Source: docs/04-test-evidence/
AT3	Automated Failover & Fencing	Region A stopped; HAProxy redispatches to B; DR controller fences A (`MAINT`); RTO measured at **3448 ms**.	**PASS**	`m2-controller.md` Source: docs/04-test-evidence/
AT4	Integrity Failback Gate	Region A restarted; held in `MAINT` until fast listing parity matches; failback to `HEALTHY` in **1.2s**.	**PASS**	`m3-failback-rpo.md` Source: docs/04-test-evidence/
AT5	GitOps Deploy & Auto-Rollback	Valid config deployed in **8.0s**; faulty config detected by health gate and rolled back in **19.0s**.	**PASS**	`m4-gitops.md` Source: docs/04-test-evidence/
AT6	Resilience Under Burst & Freeze	Crash RTO median **4732 ms**; pause RTO median **7192 ms**; 0 missing writes across all iterations.	**PASS**	`m5-chaos.md` Source: docs/04-test-evidence/
AT7	Degraded Network & Partition	500ms latency replication lag: **3.05s**; one-way partition catch-up: **4.91s**; Last-Write-Wins verified.	**PASS**	`m5-chaos.md` Source: docs/04-test-evidence/

Independent Recovery Point Objective (RPO) Measurement

RPO was empirically evaluated by issuing rapid numbered write streams (`write-00001.txt` … `write-00012.txt`) through the gateway while simultaneously injecting catastrophic failures:

- **Graceful Stop RPO:** 12 writes acknowledged; 12 writes found on Region B upon failover (`missing: []` , `rpo_objects: 0`). Stop duration: 8156 ms.
- **Abrupt SIGKILL RPO:** 12 writes acknowledged; 12 writes found on Region B upon failover (`missing: []` , `rpo_objects: 0`). Kill duration: 8641 ms.
- **Source Citation:** `docs/04-test-evidence/m3-failback-rpo.md` and `demo/demo-run.log` .

10. Chaos Engineering Resilience Suite (E1–E5)

The chaos engineering framework (`chaos/runner.py`) validates platform behavior against explicit, falsifiable hypotheses under real concurrent client write workloads (30–42 req/s):

Exp	Failure Mode	Falsifiable Hypothesis	Empirical Measured Outcome	RTO (Min/Med/Max)
E1	Abrupt Crash (SIGKILL minio-a)	First write via B succeeds ≤ 10s; RPO_failover ≤ 2; RPO_after_return == 0 under 35–42 req/s load.	**Hypothesis HELD**. Zero dropped acknowledged writes; failback clean. Load: 37.2 req/s.	2764 ms / **4732 ms** / 4787 ms
E2	Gray Failure (docker pause)	First write via B succeeds ≤ 10s; RPO_failover ≤ 2; RPO_after_return == 0 with 3.5s client timeout.	**Hypothesis HELD**. Client timeout (3.5s) + HAProxy connect (2s) completed gracefully. Rate: 28.5 req/s.	7169 ms / **7192 ms** / 7214 ms
E3	WAN Degradation (Toxiproxy)	Replication lag increases measurably under 500ms latency & 32KB/s bandwidth; 0 writes lost.	**Hypothesis HELD**. Baseline lag: 0.73s; 500ms latency lag: 3.05s; 32KB/s lag: 2.39s; 0 lost.	N/A (Replication lag tested)
E4	One-Way Partition & LWW Conflict	During A→B cut, writes to both sites persist; conflicting key resolves via LWW once healed; catch-up ≤ 15s.	**Hypothesis HELD**. 20 partition writes synced in 4.91s; latest timestamp won on conflict.	N/A (Partition test)
E5	Deploy Chaos (Kill mid-deploy)	Killing gateway during active health gate triggers auto-rollback to last_good within 35s.	**Hypothesis HELD**. Candidate 6f05e54 failed health gate; stack restored in 16.0s.	N/A (Rollback: 16.0s)

Key Insights from Chaos Experiments

- **Crash vs. Gray Failure Recovery:** SIGKILL terminates the process immediately, causing the OS kernel to emit TCP RST packets. HAProxy detects connection reset in <1 second. In contrast, `docker pause` freezes container execution without tearing down sockets. In-flight requests must wait out the client timeout (3.5s) and HAProxy's connect timeout (2.0s), explaining why E2 RTO (~7.2s) is consistently longer than E1 RTO (~4.7s).
- **Experiment E3 Lag Differences:** In Experiment E3, measured replication lag under combined latency + bandwidth (2.21s) was slightly lower than latency-only (3.05s). These variations were within run-to-run timing noise; the underlying cause was not investigated, and no causal trend is claimed.
- **Experiment E4 Partition Topology:** Because Toxiproxy sits specifically on the `site-a -> site-b` replication path (port 9002), cutting traffic severed A→B replication while B→A remained direct. This was an **asymmetric one-way partition**. All 20 non-conflicting writes synchronized within 4.91s upon link restoration.
- **Source Citation:** `docs/04-test-evidence/m5-chaos.md` .

11. Cryptographic Deep Audit & Demo Timings

Byte-for-Byte Cryptographic Payload Verification

To provide undeniable mathematical proof that no silent data corruption (bit rot) or object drift occurred across any of the disaster recovery drills, failover events, or chaos experiments, I executed a deep cryptographic audit across the entire storage estate:

```
$ python3 controller/proof.py compare --deep
{
  "match": true,
  "count_a": 149,
  "count_b": 149,
  "only_a": [],
  "only_b": [],
  "different": [],
  "sha256_checked": 149,
  "mode": "deep"
}
```

> **Audit Findings:**
> - Total Objects in Region A: **149** (across `demo` and `rpo` buckets)
> - Total Objects in Region B: **149**
> - Payload Hashes Computed: **149** full SHA-256 digests streamed from both endpoints
> - Mismatches or Corrupted Objects: **0** (`different: []`)
> - Result: **100% Cryptographic Parity Verified**.
> - Source: `docs/04-test-evidence/m6-deep-audit.md` .

Consecutive `make demo` Execution Timings

The complete end-to-end self-healing demonstration was executed multiple times consecutively on the main platform stack to confirm repeatability:

Execution Run	Duration	Exit Code	Failover RTO	Fast Gate Parity	GitOps Auto-Rollback
Run 1	1m 49s	0	4011 ms	MATCH (151 objects)	PASS (19.0s)
Run 2	1m 54s	0	3982 ms	MATCH (151 objects)	PASS (19.0s)
Run 3 (demo-run.log)	1m 51s	0	4011 ms	MATCH (151 objects)	PASS (19.0s)

Source: `demo/demo-run.log` and test execution records.

12. Mandatory Scope & System Limitations

To maintain rigorous engineering honesty, the following constraints and physical boundaries of this platform are explicitly documented:

1. Eventual Consistency & Last-Write-Wins (LWW)

MinIO site replication is an active-active, eventually consistent distributed storage engine. In scenarios where an un-fenced network partition occurs and concurrent dual-primary writes are issued to identical object keys, MinIO resolves the conflict strictly by object timestamp (Last-Write-Wins). The older version is silently overwritten. Application-level semantic conflict merging is not supported.

2. RPO Measurement Scope & Disk-Loss Boundary

The measured RPO of `0` missing objects was empirically validated under workloads of approximately 35–42 writes/sec with small objects (4 KB – 20 KB) and with **surviving disk volumes**. This demonstrates replication throughput and socket reliability under test conditions, not a mathematical zero-data-loss guarantee. Physical storage destruction or sudden disk wipe (catastrophic disk destruction) was not tested.

3. RTO Includes Client Timeouts

During gray failures (such as `docker pause` in Experiment E2), RTO measurements include the client HTTP request timeout (3.5 seconds) in addition to HAProxy's 2-second connection timeout and redispatch window, resulting in measured RTO clustering around ~7.2 seconds.

4. Fast Failback Gate vs. Deep Cryptographic Audit

The automated failback gate (`compare_sites` in routine operation) evaluates object listings, sizes, and ETags only (`sha256_checked: 0`) to achieve sub-2-second failback evaluation (1.2s across 140+ objects). Full cryptographic payload streaming and SHA-256 digest computation across every stored object is performed separately via `make audit` / `--deep` (`sha256_checked: 149`).

5. Experiment E3 Run-to-Run Noise

In Experiment E3, measured replication lag under combined latency + bandwidth (2.21s) was lower than the latency-only scenario (3.05s). These differences were within run-to-run timing noise, and the specific underlying cause was not investigated. No trend or causal relationship is claimed.

6. Asymmetric One-Way Partition (E4 Toxiproxy Placement)

Toxiproxy sits specifically on the `site-a -> site-b` replication path (port 9002). In Experiment E4, timeout toxics severed traffic from `site-a` to `site-b` only. `site-b` 's replication endpoint to `site-a` remained direct (`http://minio-a:9000`), meaning writes to `site-b` continued to reach `site-a` during the cut. This was a **one-way (asymmetric) partition**, not a complete bidirectional severance.

7. Single-Machine Simulation

Both "regions" execute as Docker containers on a single host Linux kernel. Inter-region WAN characteristics (propagation delay, packet loss, bandwidth caps) were simulated programmatically via Toxiproxy rather than traversing actual cross-continental fiber.

13. Engineering Challenges & What I Learned

Real Bugs Found and Corrected

- **First-Run Replication Bootstrap Timing:** In Milestone M0, initial site replication configuration failed intermittently because `minio-a` attempted to join `minio-b` before MinIO's internal server pool was fully initialized. Fixed by implementing an active polling readiness loop that verifies HTTP 200 on both backends before issuing `mc admin replicate add`.
- **Throwaway Test Project-Name Collisions:** When testing isolated volume lifecycle scripts, running `docker compose` without explicit project scoping reused the default project network and named volumes. Resolved by enforcing strict `COMPOSE_PROJECT_NAME=setu-fresh-test` and isolated volume naming.
- **Demo Pipeline Exit Code Masking:** When piping terminal output through `tee` in `scripts/demo.sh`, bash masked non-zero exit codes from failed sub-steps. Fixed by capturing `${PIPESTATUS[0]}` and asserting positive exit status on every critical gate.
- **Gray Failure Client Socket Deadlock:** Paused containers do not emit TCP RST packets. Client workers with short timeouts deadlocked waiting on stale sockets. Resolved by configuring explicit 3.5s client timeouts and tuning HAProxy connect timeouts.
- **Correction of Overstated Early Metrics:** In exploratory chaos runs, Experiment E2's hypothesis claimed pause RTO would exceed kill RTO, but measured values varied. Rather than forcing hardcoded pass criteria, the evaluation harness was refactored so that every assertion evaluates real, explicit clauses that can fail, ensuring honest scientific reporting.

Core Systems Engineering Takeaways

1. **Crash vs. Gray Failure Semantics:** A hard process termination (SIGKILL) is clean from a networking standpoint because the operating system kernel immediately tears down open file descriptors and emits TCP RST packets. A gray failure (socket freeze / process freeze) is far more hazardous in distributed systems because sockets remain in `ESTABLISHED` state, forcing reliance on application and proxy connect timeouts.
2. **Ingress Fencing is Essential:** Restoring a recovered primary without administrative ingress fencing creates severe split-brain hazards. Fencing primary in HAProxy (`MAINT`) until cryptographic parity is proven is the only safe method to prevent silent data overwrite.
3. **Targeted GitOps Re-creation:** In infrastructure managing persistent state, blanket `--force-recreate` must never be used. Computing diffs to recreate only modified stateless containers (e.g. gateway) prevents unnecessary storage outages.
4. **User-Space Fault Injection:** Programmatic TCP proxying (via Toxiproxy) provides far greater portability and safety than host kernel manipulation (`tc netem`), avoiding root requirements and privilege leaks.

14. Future Directions & Demonstration Materials

Future Architectural Extensions

- **Catastrophic Disk-Loss Testing:** Extending the chaos suite to simulate unrecoverable physical disk destruction (purging `minio_a_data` entirely) and measuring recovery time from cold backup volumes.
- **Multi-Host Bare-Metal Deployment:** Migrating the platform from a single WSL2 host to two physically separated Linux servers connected via a real wide-area network (e.g., WireGuard VPN over internet).
- **Kubernetes Operator Variant:** Packaging the DR state machine and GitOps reconciler as a custom Kubernetes Operator with Custom Resource Definitions (CRDs) for enterprise adoption.
- **Prometheus & OpenTelemetry Observability:** Integrating Prometheus metrics scraping and Grafana dashboards for real-time visualization of replication lag, HAProxy queue depth, and failover latency.

Demonstration Materials & Live Review Protocol

> **Live Demonstration Notice:**
> Live demonstration will be given at the Zoho Campus Offline Review (November 2026).
> A recorded terminal run of `make demo` (concluding with exit code 0) is included directly in the submission package as `demo/demo-run.log` .

For evaluators inspecting the terminal run log, the table below maps the 10 sequential demonstration steps to the real execution log:

Step #	Demonstration Action	Observed Behavior in Log	Timestamp / Duration
Step 1	Stack Health Verification	Region A, Region B, and Gateway all report UP (HTTP 200).	T+0s (Immediate)
Step 2	Normal Operation Write	Write to GW; read from B; SHA-256 match confirmed.	T+8s
Step 3	Start DR Controller	Controller initializes in HEALTHY state (PID 72085).	T+11s
Step 4	Inject Disaster (Kill A)	Region A stopped; HAProxy redispatches to Region B.	T+13s
Step 5	Failover & Fencing	Controller enters FAILOVER; fences A (MAINT); RTO: **4011 ms**.	T+18s
Step 6	Recovery & Failback Gate	Region A restarted; held in MAINT; synced; un-fenced (UP).	T+32s (14s sync)
Step 7 & 8	Independent RPO Measure	Numbered write stream under stop & kill: **0 missing**.	T+45s
Step 9	Ephemeral Bucket Cleanup	Temporary RPO test buckets cleanly deleted.	T+52s
Step 10	GitOps Deploy & Rollback	Valid commit in 8s; bad commit rolled back in **19s**.	T+1m 15s – 1m 51s

15. Appendix: Command Cheat Sheet & Glossary

Operator Command Cheat Sheet

Command	Operation Performed	Target Outcome
`make doctor`	Validates system dependencies, Docker, Python 3, and memory allocation.	Pre-flight validation (0 errors).
`make up`	Starts all platform containers (minio-a, minio-b, gateway, toxiproxy, monitor).	Multi-region stack UP.
`make health`	Queries HTTP 200 health status across all endpoints.	Reports status of all nodes.
`make demo`	Executes complete 10-step end-to-end self-healing demonstration.	Proves AT1–AT5 in ~1m50s (Exit code 0).
`make chaos`	Executes automated 5-experiment chaos engineering resilience suite.	Generates `m5-chaos.md`.
`make audit`	Executes deep cryptographic SHA-256 payload audit across all objects.	Outputs JSON confirmation of parity.
`make package`	Builds final submission directory and verified archive.	Produces verified `.zip` (<20 MB).
`make down`	Stops platform containers while safely preserving volume data.	Clean stack shutdown.

Technical Glossary

- **RTO (Recovery Time Objective):** The elapsed duration between primary failure detection and the first successful write acknowledged on the standby region.
- **RPO (Recovery Point Objective):** The quantity of data lost during disaster, measured here as acknowledged client writes missing from the backup region.
- **Failback:** The controlled process of restoring primary traffic to Region A after resynchronization is mathematically confirmed.
- **Administrative Fencing:** Forcibly locking a failed or recovering node into `MAINT` state on HAProxy's admin socket to prevent split-brain dual-primary writing.
- **Site Replication:** MinIO's active-active bucket and object synchronization mechanism operating over S3 HTTP APIs.
- **GitOps Continuous Delivery:** Declarative infrastructure deployment where git commits drive container convergence with automated pre-flight checks and rollback.
- **Chaos Engineering:** The discipline of experimenting on a distributed system to build confidence in its capability to withstand turbulent conditions.

> **Technical Defense Reference:**
> For detailed technical answers to 15 key architectural and distributed systems questions (including split-brain prevention, Nginx vs HAProxy tradeoffs, Toxiproxy user-space mechanics, and SigV4 signer architecture), refer to `docs/06-reviewer-qa.md`.